Computational Control

Lecture Handout

March 15, 2026

Contents

1 Dynamic Programming and LQR

In general we formulate the optimization control task as

$$\min_{\mathbf{u}\in\mathcal{U}} J(\mathbf{u}) \tag{1}$$

where $\mathcal{U} = \tilde{\mathcal{U}} \times \{0, 1, \dots, \mathrm{T}\}$ is the set of the feasible inputs and $J(\mathbf{u})$ is generally the sum of some *stage costs* and a terminal cost/constraint.
The issue is that generally 1 is an intractable optimization problem. This is due to several factors:

- the dimension of $\mathbf{u}$ is too big;
- the cost function requires an accurate model of the system;
- the optimization problem is **non-convex**;
- In presence of **disturbances** and **uncertainity** the open-loop nature of the task makes it useless

In this context, Dynamic Programming (DP) is a powerfull tool whenever we can decompose the problem. In particular when the problem allows for a **Markovian representation**:

Key assumptions: Markovian representation

The following hypothesis are crucial in order for DP to be meaningfull:

1. There exists a **Markovian State**[a] that evolves according to some dynamics
$$x_{t+1} = f_t(x_t, u_t)^{[b]}$$
2. The initial condition x_0 is known.
3. The cost is additive over time, i.e.
$$J = \sum_t g_t(x_t, u_t)$$

[a] i.e. $X_{t+1} \perp X_{1:t-1} | X_t$
[b] discrete time dynamical system representation. Notice f_t can change with time

In this context a powerfull result is given by **Bellman's principle**. Considering the following setup:

$$\begin{aligned} \min_{\mathbf{u},\mathbf{x}} \quad & \sum_{t=0}^{T} g_t(x_t, u_t) \\ \text{subject to} \quad & x_{t+1} = f_t(x_t, u_t) \\ & x_0 = X_0 \\ & x_t \in \mathcal{X}_t \ \forall t \\ & u_t \in \mathcal{U}_t \ \forall t. \end{aligned}$$

a very powerfull result on which relies DP is **Bellman's optimality principle**.

Bellman's optimality principle

An optimal policy has the property that whatever the initial state and the initial decisions are, the remaining decisions must consititute an optimal policy with regard to the state resulting from the first decision.

In a intuitive way we can say that any tail of an optimal trajectory is optimal too.

$$\begin{aligned} V_0(X_0) &= \min_{u_0} \left\{ g_0(X_0, u_0) + \min_{\substack{x_1,\dots,x_T \\ u_1,\dots,u_T}} \sum_{t=1}^{T} g_t(x_t, u_t) \;\text{ s.t. } \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_1 = f_0(X_0, u_0) \end{cases} \right\} \\ &= \min_{u_0} \{ g_0(X_0, u_0) + V_1(x_1) \} \\ &= \min_{u_0} \{ g_0(X_0, u_0) + V_1(f_0(x_0, u_0)) \} \end{aligned}$$

We can see that the problems are nested, therefore we need to proceed in a **backward** way, starting from the last time step and going backward in time. In particular we need to proceed in the following order

1. solve the subproblem parametrized by x_1;
2. compute the value function V_1;
3. solve the outer problem
$$\min_{u_0} \{ g(X_0, u_0) + V_1(f_0(X_0, u_0)) \}$$

1.1 Dynamic Programming

We can now write the general DP algorithm, which is a backward procedure to compute the value function and the optimal policy. We start from the last time step and we proceed backward in time until we reach the initial time step.
At the last stage (T) the subproblem is trivial, since there are no more decisions to be taken, therefore we have

$$V_T(x_T) = g_T(x_T)$$

At time $T-1$ we compute the value function as

$$\min_{u_{T-1}} \{ g_{T-1}(x_{T-1}, u_{T-1}) + V_T(f_{T-1}(x_{T-1}, u_{T-1})) \}$$

In particular notice that in the formulation above we know the value of $V_T(x_T)$, therefore we can compute the value function at time $T-1$ and so on until we reach the initial time step. So at this moment we have an optimal policy for the time step $T-1$, that means that we know how to compute the optimal control action at time $T-1$ given the state at time $T-1$. We can repeat this procedure until we reach the initial time step, therefore we have an optimal policy for all time steps.
At time $T-2$ we compute $V_{T-2}(x_{T-2})$ as

$$\min_{u_{T-2}} \{ g_{T-2}(x_{T-2}, u_{T-2}) + V_{T-1}(f_{T-2}(x_{T-2}, u_{T-2})) \}$$

Notice that this value function $V_{T-2}(x_{T-2})$ is a function of the state x_{T-2}, therefore we have to compute it $\forall x_{T-2} \in \mathcal{X}_{T-2}$, which is generally a very hard task.

Problem Decomposition

We have seen that the optimal control problem is decomposed into **stage problems** that we can solve via **backward induction** :

$$V_t(x_t) = \min_{u_t} \{ g_t(x_t, u_t) + V_{t+1}(f_t(x_t, u_t)) \} \tag{2}$$

And $V_T(x_T) = g_T(x_T)$

A reasonable question at this point would be, in which cases is this stage problem formulation simple to solve ? We can list three scenarios:

- Decision space $\tilde{\mathcal{U}}$ is convec , or maybe finite.
- If g_t is convex in u_t.
- If V_{t+1} evaluated at the future step f_t is convex in u_t.

In practice, a very common setup in which the assumptions above are satisfied is when we have a **quadratic cost** g_t and a **linear dynamics** f_t, which is the so called **LQR problem**. Another way is also to use approximations of the value function, in which case maybe not all the assumptions above are satisfied, but we can still solve the stage problem.

1.2 Optimal LQR

The LQR problem is a special case of the optimal control problem, in which we have a linear dynamics and a quadratic cost. In particular we have:

- Markovian update and linear dynamics

$$x_{t+1} = A_t x_t + B_t u_t$$

- Quadratic cost function:

$$\sum_{t=0}^{T-1} \left(x_t^\top \, Q_t \, x_t + u_t^\top \, R_t \, u_t \right) + x_T^\top \, S \, x_T \quad \text{with } Q, S \geq 0 \,,\, R > 0$$

- Value function (i.e. minimum cost to go starting from x at time t) :

$$V_t(x) = \min_{u_t, \ldots, u_{T-1}} \sum_{k=t}^{T-1} \left(x_k^\top \, Q_k \, x_k + u_k^\top \, R_k \, u_k \right) + x_T^\top \, S \, x_T$$

$$\begin{aligned} \text{subject to } & x_{k+1} = A_k x_k + B_k u_k \quad \forall k = t, \ldots, T-1 \\ & x_t = x \end{aligned}$$

We say that the optimal cost is $V_0(x_0)$.

1.3 Dynamic Programming for LQR

Let us now specialize the dynamic programming recursion to the Linear Quadratic Regulator (LQR) problem. Recall that the value function represents the optimal *cost-to-go*, i.e. the minimum cost achievable from a given state when acting optimally from that point onward.

A key observation in the LQR setting is that the terminal cost is quadratic in the state. Indeed, from the problem formulation we have

$$V_T(x) = x^\top S x,$$

which is a convex quadratic function since $S \succeq 0$.

The main idea of the LQR derivation is to show that the value function $V_t(x)$ is also quadratic and has the form of $V_t(x) = x^\top \, P_t \, x$ for some symmetric positive semidefinite matrix P_t.
We will then show that the matrices P_t can then be computed recursively by working *backward in time* starting from the terminal time T. Eventually the the optimal control input u_t will be computed as a solution of a convex quadratic optimization problem, which admits a closed-form solution.

To derive the recursion, consider the dynamic programming stage problem at time t where the value function is

$$V_t(x) = \min_u \Big(x^\top Q\, x + u^\top R\, u + V_{t+1}(Ax + Bu) \Big).$$

We now proceed by induction, assuming that $V_{t+1}(x) = x^\top P_{t+1} x$ for some positive semidefinite matrix P_{t+1}. Then $V_t(x)$ can be written as

$$\begin{aligned} V_t(x) &= x^\top Q x + \min_u \Big(u^\top R u + (Ax + Bu)^\top P_{t+1}(Ax + Bu) \Big) \\ &= x^\top Q x + \min_u \Big(u^\top R u + x^\top A^\top P_{t+1} A x + 2u^\top B^\top P_{t+1} A x + u^\top B^\top P_{t+1} B u \Big) \\ &= x^\top\, Q\, x + \min_u \Big(u^\top (R + B^\top P_{t+1} B) u + x^\top A^\top P_{t+1} A x + 2u^\top B^\top P_{t+1} A x \Big). \end{aligned}$$

We set the gradient of the expression inside the minimization with respect to u to zero:

$$(R + B^\top P_{t+1} B)u + B^\top P_{t+1} A x = 0$$

Yielding the optimal control input

$$\boxed{u^* = -(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x}$$

But now we need to check that $V_t(x)$ is indeed quadratic in X. That means $V_t(x) = x^\top P_t x$ for some P_t and we have to find how to compute P_t. We can do that by plugging the optimal control input u^* back into the expression for $V_t(x)$:

$$\begin{aligned} V_t(x) &= x^\top Q x + (u^*)^\top R u^* + (Ax + Bu^*)^\top P_{t+1}(Ax + Bu^*) \\ &= x^\top Q x + (u^*)^\top R u^* + x^\top A^\top P_{t+1} A x + 2(u^*)^\top B^\top P_{t+1} A x + (u^*)^\top B^\top P_{t+1} B u^* \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + (u^*)^\top (R + B^\top P_{t+1} B) u^* + 2(u^*)^\top B^\top P_{t+1} A x \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + (u^*)^\top \big((R + B^\top P_{t+1} B) u^* + 2B^\top P_{t+1} A x \big) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x \\ &\quad -((R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x)^\top (-(R + B^\top P_{t+1} B)(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x + 2B^\top P_{t+1} A x) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + -((R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x)^\top (B^\top P_{t+1} A x) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x - x^\top A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x \\ &= x^\top \Big(Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \Big) x \\ &= x^\top P_t x \quad \square \end{aligned}$$

To complete the proof one should check that P_t is positive semidefinite. Here we homit the proof.
In cocnlusion we have been able to derive a closed-form expression for the optimal control input u^* for the LQR problem.Be aware that this solution is not valid anymore if the problem becomes constrained.

We summerize here the main results:

Optimal LQR solution

Backward induction

$$u_t = \underbrace{-(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A}_{\Gamma_t} \, x_t$$

where

$$P_t = Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \quad \text{and } P_T = S$$

Forward integration from x_0:

$$x_{t+1} = Ax_t + Bu_t$$

So clearly one could compute this whole open-loop sequence $u_0, \ldots, u_{T-1}$ in a offline computation. But what if the optimal control input **u** takes us to a trajectory different from the one we compouted ? In this case we can use the optimal policy Γ_t to compute the optimal control input at each time step, given the current state x_t. This is the main advantage of having a closed-form solution for the optimal control input, since we can use it in a feedback way.

We can do a quick comparison of the computational cost of using these equations in a feedback way vs computing the whole open-loop sequence. By fixing m as the size of the states and n as the size of the control input, we have the following computational costs:

- **Offline computation**
  - $n \times n$ matrix multiplications to compute P_t for $t = T-1, \ldots, 0$
  - $m \times m$ matrix inversions to compute Γ_t for $t = T-1, \ldots, 0$
  - T iterations to compute P_t and Γ_t for $t = T-1, \ldots, 0$
- **Online computation**
  - Storage of T $m \times n$ matrices Γ_t for $t = T-1, \ldots, 0$
  - $m \times m$ matrix multiplications to compute u_t at each time step, given x_t

1.3.1 Infinite-Horizon LQR

In many practical control applications the system is required to operate over a very long time horizon. Examples include regulation problems where the controller must stabilize a system indefinitely, or tracking tasks where disturbances may occur continuously.

For these reasons it is natural to study the limit case where the horizon tends to infinity. The resulting problem is known as the **infinite-horizon Linear Quadratic Regulator (LQR)**.

Consider the linear time-invariant system

$$x_{t+1} = Ax_t + Bu_t, \qquad x_0 \text{ given}$$

and the infinite-horizon quadratic cost

$$\min_{u_0, u_1, \ldots} \sum_{t=0}^{\infty} \left(x_t^\top Q x_t + u_t^\top R u_t \right), \qquad Q \succeq 0, \; R \succ 0.$$

Compared to the finite-horizon case, the cost now accumulates indefinitely. Therefore a first important question concerns the **feasibility** of the problem.

Feasibility

If the pair (A, B) is **stabilizable**, then there exists a control input sequence that yields a finite value of the infinite-horizon cost.

Indeed, if the system is stabilizable there exists a feedback matrix K such that the closed-loop matrix $A + BK$ has all eigenvalues inside the unit circle. Consider the linear feedback policy

$$u_t = Kx_t.$$

The state then evolves according to

$$x_t = (A + BK)^t x_0.$$

Substituting this into the cost yields

$$V(x_0) = \sum_{t=0}^{\infty} \left(x_t^\top Q x_t + x_t^\top K^\top R K x_t \right).$$

Using the closed-loop dynamics we obtain

$$V(x_0) = x_0^\top \left(\sum_{t=0}^{\infty} (A + BK)^{t\top} (Q + K^\top R K)(A + BK)^t \right) x_0.$$

Since $A + BK$ is stable, the above summation behaves like a *matrix geometric series* and therefore converges. Consequently the cost is finite.

But how do we compute the optimal policy for the infinite-horizon LQR problem ? The answer is that we can still apply the dynamic programming recursion, but now the value function does not depend on time. The solution is given by the **Discrete Algebraic Riccati Equation (DARE)**, which is a fixed-point equation for the matrix P that defines the value function.

Algebraic Riccati Equation

The iteration

$$P_{t-1} = Q + A^\top P_t A - A^\top P_t B (R + B^\top P_t B)^{-1} B^\top P_t A \qquad P_0 = 0$$

converges, for $t \to -\infty$, to a unique positive semidefinite solution $P_\infty \succ 0$ of the algebraic Riccati equation

$$P = Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A$$

We can show that then the optimal feedback control in the infinite-horizon case is given by

$$u_t = \underbrace{-(R + B^\top P B)^{-1} B^\top P A}_{F_\infty} x_t$$

and thus minimizes the infinite horizon cost and yields $V(X_0) = X_0^\top P X_0$.
We now show again a comparison of the computational cost of using the infinite-horizon solution in a feedback way vs computing the whole open-loop sequence. By fixing m as the size of the states and n as the size of the control input, we have the following computational costs:

- **Offline computation**
  - $n \times n$ matrix multiplications to compute P
  - $m \times m$ matrix inversions to compute F_∞

  - T iterations to compute P and F_∞ for $t = T-1, \dots, 0$
  - "Infinite iterations" to compute P for $t \to -\infty$[1]

- **Online computation**
  - Storage of a **single** $m \times n$ matrix F_∞. So less memory is required compared to the finite-horizon case, since we do not need to store T matrices Γ_t.
  - $m \times n$ matrix multiplications to compute u_t at each time step, given x_t

A natural question at this point is whether the optimal infinite-horizon feedback law is also *stabilizing*. Indeed, even if the infinite-horizon cost is well defined and the Riccati equation admits a solution, one would still like to know whether the resulting closed-loop dynamics

$$x_{t+1} = (A + BF_\infty)x_t$$

are asymptotically stable.

This is a meaningful question because the quadratic cost does *not* necessarily penalize all state directions. In particular, if some unstable state component is not "seen" by the cost, then the optimizer may have no incentive to stabilize it.

Illustrative example Consider the system

$$x_{t+1} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} x_t + u_t, \qquad Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad R = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Here both open-loop modes are unstable, since the eigenvalues of A are 2 and 3. However, the stage cost penalizes only the first state component x_1, while the second component x_2 does not appear in the state cost. Therefore, from the viewpoint of the objective function, an unstable behavior in the second state may go completely unnoticed unless it is indirectly forced to be controlled through the dynamics.

This already suggests an important principle:

> Unstable modes must be visible in the cost function, otherwise the optimal controller may fail to stabilize them.

To make this statement precise, write $Q = C^\top C$. Then the cost penalizes exactly the directions that are observable through the pair (C, A).

Stability of the optimal infinite-horizon LQR controller

Let $Q = C^\top C$. The optimal infinite-horizon feedback F_∞ stabilizes the system if and only if

1. the pair (A, B) is stabilizable;
2. the pair (C, A) has no unobservable unstable modes.

Equivalently, all unstable modes of A must be detectable through the state penalty Q.

The second condition is often stated by saying that the pair $(A, Q^{1/2})$ must be **detectable**. Detectability is weaker than observability: it does not require all modes to be observable, but only the unstable ones.

[1] P_∞ can be computed by solving the Algebraic Riccati Equation

Interpretation This theorem has a clear practical meaning. The matrix R penalizes the control effort, while Q determines which state directions the controller tries to regulate. If an unstable mode is not penalized by Q, then the optimization problem may prefer not to spend control effort on that mode, since doing so does not improve the objective. As a consequence, the optimal controller may exist but fail to stabilize the closed-loop system.

Hence, in practice:

- (A, B) stabilizable ensures that unstable modes can in principle be controlled;
- detectability of $(A, Q^{1/2})$ ensures that every unstable mode matters in the cost.

When both conditions hold, the stabilizing solution P of the algebraic Riccati equation yields the optimal feedback

$$u_t = F_\infty x_t, \qquad F_\infty = -(R + B^\top P B)^{-1} B^\top P A,$$

and the closed-loop matrix $A + BF_\infty$ has all eigenvalues inside the unit disk.